

# Networking Basics and Their Importance in Cyber security

In today's digital world, networking is the backbone of communication between devices. From sending a simple message to accessing secure banking systems, everything depends on how networks are designed, managed, and protected. Understanding networking basics is especially important in cyber security, because most cyber attacks target network weaknesses. Without strong networking knowledge, it becomes difficult to secure systems, detect threats, or respond to attacks effectively.

## What is Networking?

Networking is the process of connecting multiple devices such as computers, smartphones, servers, and printers so they can communicate and share data. These devices are connected using wired or wireless technologies like Ethernet cables, Wi-Fi, or fiber optics. Networking allows users to share resources, transfer information, and access services over local or global networks. In simple terms, networking works like a digital highway where data travels from one device to another.

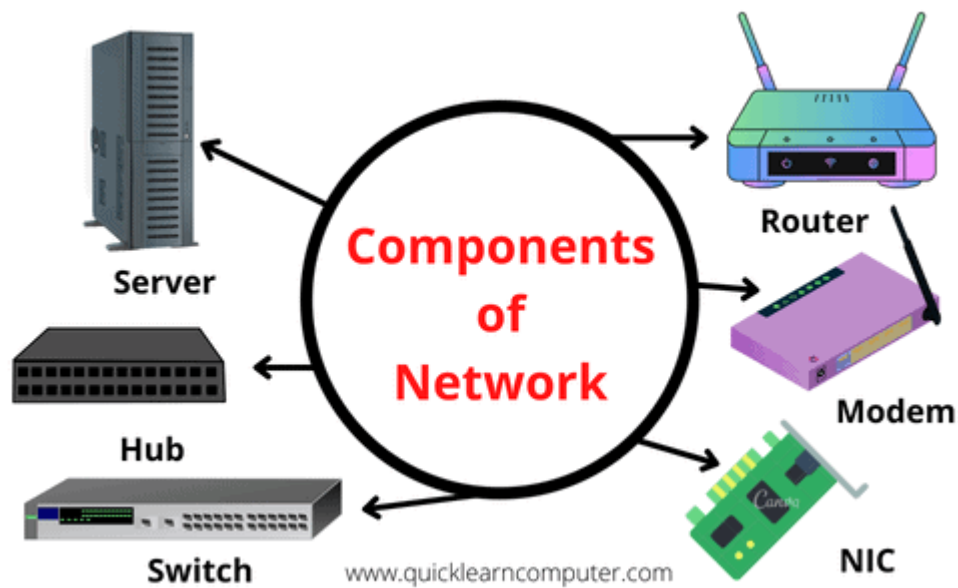
## Types of Network:

Networks are classified based on their size and purpose. Each type plays a specific role in modern communication.

A **Local Area Network (LAN)** covers a small area such as a home, office, or school. It is fast, easy to manage, and usually private. A **Wide Area Network (WAN)** connects networks over large distances, such as cities or countries. The internet itself is the biggest example of a WAN. A **Metropolitan Area Network (MAN)** covers a city or campus, while a **Personal Area Network (PAN)** connects personal devices over a short range, such as Bluetooth connections between a phone and wireless earbuds.

Understanding these network types helps cyber security professionals decide how to apply security controls at different levels.

## Network Components and Devices:



## Introduction

Understanding network components and devices is essential in today's interconnected world. From the foundational devices like routers and switches to advanced security systems and VoIP infrastructure, this article explores the key elements that make up modern networks.

## Common Network Devices

### Firewall

- A firewall is a security device that filters incoming and outgoing network traffic.
- It acts as a barrier between a trusted internal network and untrusted external networks.
- Controls traffic based on a set of security rules, permitting or blocking data packets.

### Router

- Routers connect different networks and route data packets between them.
- They determine the best path for data to travel, based on destination IP addresses.
- Enable communication between devices on different subnets or networks.

## **Switch**

- Switches operate at the data link layer and are used to connect devices within a local area network (LAN).
- They use MAC addresses to forward data to the correct device.
- Improve network efficiency by delivering data only to the intended recipient.

## **Hub**

- Hubs connect multiple devices in a LAN and broadcast data to all connected devices.
- Lacks intelligence and does not filter or discriminate data packets.
- Less efficient than switches and can lead to network congestion.

## **Bridge**

- Bridges connect and filter traffic between two or more network segments.
- Operate at the data link layer and can reduce collision domains in Ethernet networks.
- Enhance network performance and segmentation by selectively forwarding data.

## **Modems**

- Modems convert digital data from computers into analog signals for transmission over telephone lines (DSL) or cable lines.
- They also convert incoming analog signals back into digital data.
- Facilitate internet access by bridging digital devices with analog communication networks.

### **Wireless Access Point (WAP)**

- WAPs enable wireless devices to connect to a wired network.
- Serve as the central point for Wi-Fi connectivity, allowing wireless clients to access network resources.
- Essential for creating wireless LANs (WLANs) in homes and businesses.

### **Media Converter**

- Media converters adapt different types of network media, such as fiber optic and copper, to allow for network expansion.
- They can extend the reach of networks over long distances.
- Convert between various physical media to ensure compatibility and seamless data transmission.

### **Wireless Range Extender**

- Range extenders boost the coverage of Wi-Fi networks.
- They capture an existing Wi-Fi signal and retransmit it, expanding the network's range.
- Ideal for eliminating Wi-Fi dead zones and ensuring comprehensive coverage.

## VoIP Endpoint

- VoIP endpoints, like IP phones and softphones, enable voice communication over IP networks.
- They convert voice into data packets for transmission.
- Key components in Voice over IP (VoIP) telephony systems, allowing cost-effective voice communication.

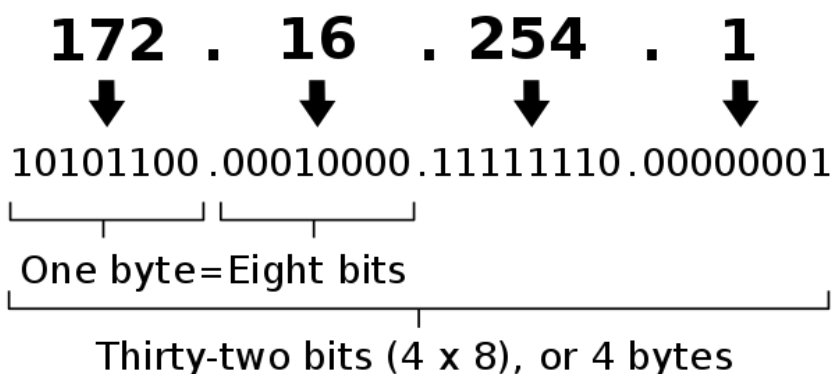
## Basics of IP Addresses in Computer Networking

Each device connected to the internet has a unique identifier. Most networks today, including all computers on the internet, use the **TCP/IP** as a standard to communicate on the network. In the TCP/IP protocol, this unique identifier is the **IP Address**. The two kinds of IP Addresses are **IPv4** and **IPv6**.

### IPv4 vs IPv6

**IPv4** uses 32 binary bits to create a single unique address on the network. An IPv4 address is expressed by four numbers separated by dots. Each number is the decimal (base-10) representation for an eight-digit binary (base-2) number, also called an octet.

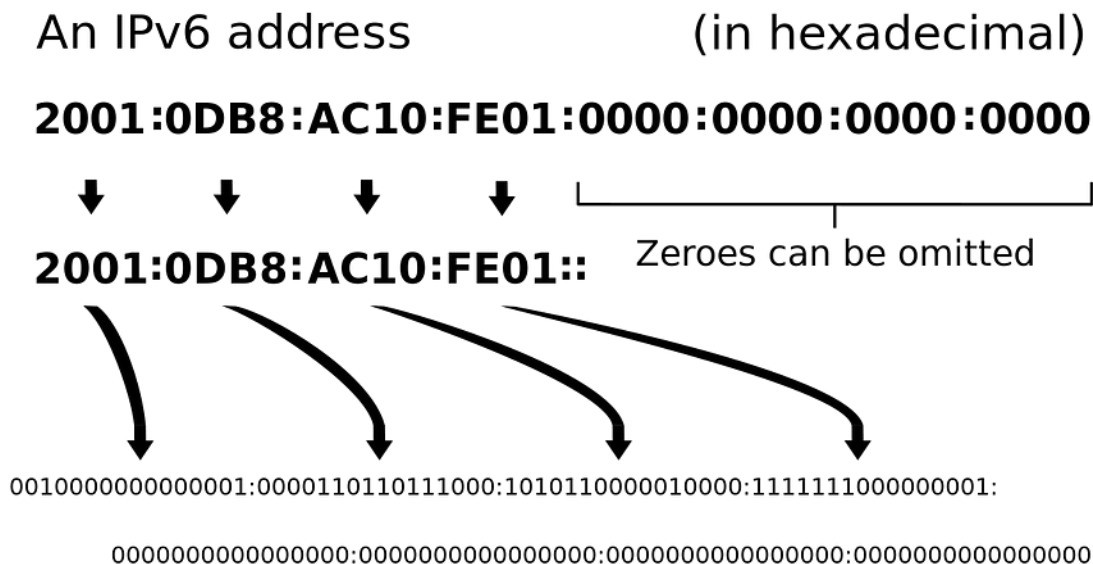
### An IPv4 address (dotted-decimal notation)



**IPv6** uses 128 binary bits to create a single unique address on the network. An IPv6 address is expressed by eight groups of hexadecimal (base-16) numbers separated by colons. Groups of

numbers that contain all zeros are often omitted to save space, leaving a colon separator to mark the gap .

Press enter or click to view image in full size

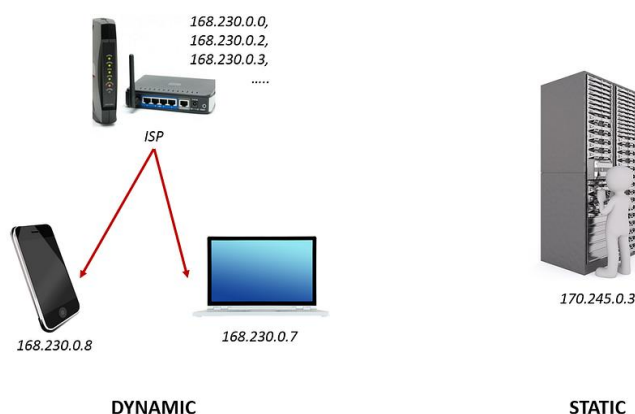


IPv6 space is much larger than the IPv4 space due the use of hexadecimal as well as having 8 groups. Most devices use IPv4. However, due to advent of IoT devices and the greater demand for IP Addresses, more and more devices are accepting IPv6.

## Static vs Dynamic

How does your computer get its IP address? An IP address can be either dynamic or static.

Press enter or click to view image in full size



**Static** address is one that you configure yourself by editing your computer's network settings. This type of address is rare, and it can create network issues if you use it without a good understanding of TCP/IP.

**Dynamic** addresses are the most common. They're assigned by the Dynamic Host Configuration Protocol (DHCP), a service running on the network. DHCP typically runs on network hardware such as **routers** or dedicated DHCP servers. Dynamic IP addresses are issued using a leasing system, meaning that the IP address is only active for a limited time. If the lease expires, the computer will automatically request a new lease.

## IP Classes

Typically, the IPv4 space allows us to have addresses between **0.0.0.0** to **255.255.255.255**. However, some numbers in that range are reserved for specific purposes on TCP/IP networks. These reservations are recognized by the authority on TCP/IP addressing, the Internet Assigned Numbers Authority (IANA). Four specific reservations include the following:

- **0.0.0.0** — This represents the default network, which is the abstract concept of just being connected to a TCP/IP network.
- **255.255.255.255** — This address is reserved for network broadcasts, or messages that should go to all computers on the network.
- **127.0.0.1** — This is called the loopback address, meaning your computer's way of identifying itself, whether or not it has an assigned IP address.
- **169.254.0.1** to **169.254.255.254** — This is the Automatic Private IP Addressing (APIPA) range of addresses assigned automatically when a computer's unsuccessful getting an address from a DHCP server.

The other IP address reservations are for **subnet classes**. A **subnet** is a smaller network of computers connected to a larger network through a router. The subnet can have its own address system so computers on the same subnet can communicate quickly without sending data across the larger network. A router on a TCP/IP network, including the Internet, is

configured to recognize one or more subnets and route network traffic appropriately. The following are the IP addresses reserved for subnets:

- **10.0.0.0 to 10.255.255.255** — This falls within the Class A address range of **1.0.0.0 to 127.0.0.0**, in which the first bit is 0.
- **172.16.0.0 to 172.31.255.255** — This falls within the Class B address range of **128.0.0.0 to 191.255.0.0**, in which the first two bits are 10.
- **192.168.0.0 to 192.168.255.255** — This falls within the Class C range of **192.0.0.0 through 223.255.255.0**, in which the first three bits are 110.
- Multicast (formerly called Class D) — The first four bits in the address are 1110, with addresses ranging from **224.0.0.0 to 239.255.255.255**.
- Reserved for future/experimental use (formerly called Class E) — addresses **240.0.0.0 to 254.255.255.254**.

The first three (within Classes A, B and C) are those most used in creating subnets. Later, we'll see how a subnet uses these addresses. The IANA has outlined specific uses for multicast addresses within Internet Engineering Task Force (IETF) document [RFC 5771](#). However, it hasn't designated a purpose or future plan for Class E addresses since it reserved the block in its 1989 document [RFC 1112](#). Before IPv6, the Internet was filled with debate about whether the IANA should release Class E for general use.

## Subnets

When you type **ipconfig** on your UNIX terminal (or CMD prompt for Windows users), you will get a pretty detailed display of your IP Address information. I have taken a screenshot of the section that I am interested in.

Press enter or click to view image in full size



```
Wireless LAN adapter Wireless Network Connection:

Connection-specific DNS Suffix . : attlocal.net
IPv6 Address. . . . . : 2600:1700:7c00:5780:f575:1e82:f0a2:1ebc
Temporary IPv6 Address. . . . . : 2600:1700:7c00:5780:d15b:25f4:24b4:a1ab
Link-local IPv6 Address . . . . . : fe80::f575:1e82:f0a2:1ebc%12
IPv4 Address. . . . . : 192.168.1.69
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::e222:3ff:fe7e:4ea5%12
                          192.168.1.254
```

ipconfig partial output

- IP address: **192.168.1.69**
- Subnet mask: **255.255.255.0**
- Twenty-four bits (three octets) reserved for network identity
- Eight bits (one octet) reserved for nodes
- Subnet identity based on subnet mask (first address): **192.168.1.0**
- The reserved broadcast address for the subnet (last address): **192.168.1.255**
- Example addresses on the same network: **192.168.1.1, 192.168.1.103**
- Example addresses not on the same network: **192.168.2.1, 192.168.2.103**

IP addresses on a subnet have two parts: **network** and **node**. The network part identifies the subnet itself. The node, also called the host, is an individual piece of computer equipment connected to the network and requiring a unique address. Each computer knows how to separate the two parts of the IP address by using a **subnet mask**. A subnet mask looks somewhat like an IP address, but it's actually just a filter used to determine which part of an IP address designates the network and node.

Press enter or click to view image in full size

|                    |                 |                 |                 |               |
|--------------------|-----------------|-----------------|-----------------|---------------|
|                    | <b>Decimal</b>  |                 |                 |               |
|                    | <b>Octet</b>    |                 |                 |               |
| <b>IP Address</b>  | 192<br>11000000 | 168<br>10101000 | 1<br>00000001   | 69<br>1000101 |
| <b>Subnet Mask</b> | 255<br>11111111 | 255<br>11111111 | 255<br>11111111 | 0<br>00000000 |
|                    | Network         |                 |                 | Node          |

In the current situation, the subnet mask is 255.255.255.0, indicating that 1 byte being dedicated for the host. It can also be 255.255.0.0 (2 bytes) and 255.0.0.0 (3 bytes).

### Ports, Protocols & Services In Computer Networks ?

What's an IP Port?

Apart from all the hardware ports, an **Internet Protocol (IP)** port is a 16-bit unsigned integer, hence ranging from **0 to 65535** (total of **2 raised power 16**). Further, the assignment of all **65536 TCP/UDP ports** is maintained by the Internet Assigned Numbers Authority (IANA) & are classified into 3 main ranges:

- 1. Well-Known Ports** (Ports in range from 0 - 1023 are controlled & assigned by various well-known services)
- 2. Registered Ports** (Ports in range from 1024 - 49151 are neither controlled nor assigned, but can be registered to prevent duplication)
- 3. Dynamic/Private Ports** (Ports in range from 49152 - 65535 are not controlled, assigned or registered but used for temporary/private use)

All the ports have their respective significance & are an essential component of **Internet Protocol (IP)**. Now, to dig deep into each of the categories, let's take a look upon the protocols & services as they totally depend upon the ports & their kinds.

What we mean by Protocols & Their Respective Services?

The above 3 main types of the ports can each be classified under **Transmission Control Protocol (TCP)** as well as **User Datagram Protocol (UDP)**, under which we again have many protocols running various services. A full list of all **TCP/UDP** port number registry & their respective running services can be found [Here](#). Moreover, it's always good if we have the most common ports with their protocols & respective services in our mind so that it becomes easy to detect anomalies quickly.

Some of the most common **TCP ports** with their respective service

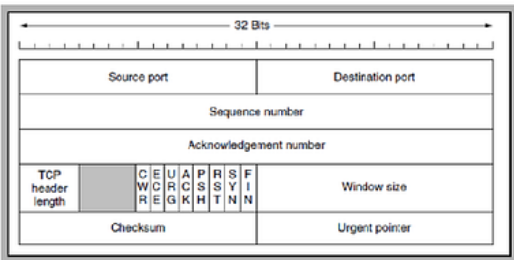
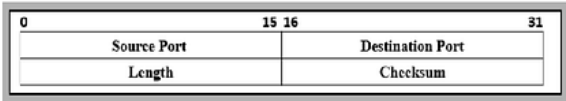
|             |      |                                            |
|-------------|------|--------------------------------------------|
| FTP         | 21   | File Transfer Protocol                     |
| SSH         | 22   | Secure Shell                               |
| Telnet      | 23   | The Telnet Service                         |
| SMTP        | 25   | Simple Mail Transfer Protocol              |
| DNS         | 53   | Domain Name Service                        |
| HTTP        | 80   | Hyper-Text Transfer Protocol               |
| Kerberos    | 88   | Kerberos Network Authentication System     |
| POP2        | 109  | Post Office Protocol Version 2             |
| POP3        | 110  | Post Office Protocol Version 3             |
| NTP         | 123  | Network Time Protocol                      |
| NETBIOS-NS  | 137  | NetBIOS Name Service                       |
| NETBIOS-DGM | 138  | NetBIOS Datagram Service                   |
| NETBIOS-SSN | 139  | NetBIOS Session Service                    |
| IMAP        | 143  | Internet Message Access Protocol           |
| SNMP        | 161  | Simple Network Management Protocol         |
| IMAPv3      | 220  | Internet Message Access Protocol Version 3 |
| HTTPS       | 443  | Secure Hyper-Text Transfer Protocol        |
| Kerberos-DS | 445  | Server Message Block (SMB)                 |
| SOCKS       | 1080 | SOCKS Network Application Proxy Services   |
| NFS         | 2049 | Network File System                        |
| MySQL       | 3306 | MySQL Database Service                     |

Similarly, Most of these run on **UDP** as well (Like *DNS, HTTP, Kerberos, NTP, NetBIOS-NS, NetBIOS-DGM, NetBIOS-SSN, IMAP, SNMP, IMAPv3, Kerberos-DS (SMB), SOCKS & NFS*). Apart from these, we do have many of them. But these set the basic floor for the whole

building, full of windows & doors, representing different ports & protocols running respective services.

The overall **difference** between **TCP & UDP** can be precisely categorized with the help of below comparison:

Press enter or click to view image in full size

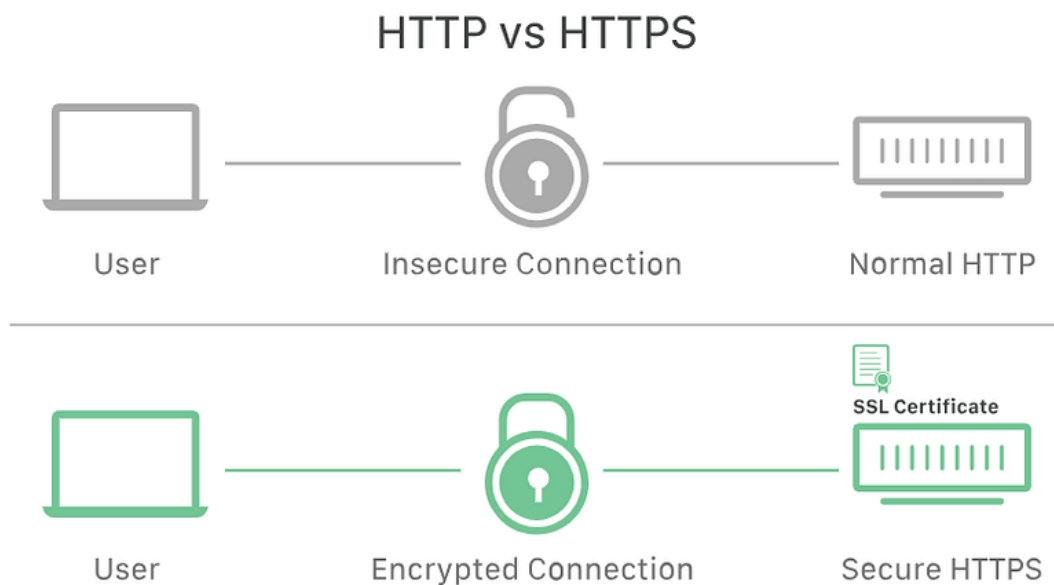
| Transmission Control Protocol (TCP)                                                                   | User Datagram Protocol (UDP)                                                                           |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| - Connection-oriented Communication Protocol.                                                         | - Connectionless Communication Protocol.                                                               |
| - Data Units known as 'Packets'.                                                                      | - Data Units Commonly Referred as Datagrams.                                                           |
| - Rearranges Data Packets to be Received in their Intended Order.                                     | - Sends Datagrams Independently, meaning that they may Arrive in a Different Order.                    |
| - Uses a 3-Way Handshake to Establish Reliable Connections.                                           | - Doesn't Use a Handshake Process.                                                                     |
| - Confirms Data Delivery Receipts.                                                                    | - Doesn't Use Delivery Receipts.                                                                       |
| - Results in Higher Latency.                                                                          | - Designed for Faster Data Transmission.                                                               |
| - Guarantees Data Delivery by Prioritizing Data Integrity, Completeness, & Reliability.               | - Prioritizes Speed & Often Results in Data Loss.                                                      |
| - Ideal for Reliable Data Transmissions.                                                              | - Ideal for Real-Time Data Transfers & Streaming to Avoid Delay.                                       |
|  <p>TCP Header</p> |  <p>UDP Header</p> |

So, till now, it must be clear that what we mean by “**Ports**”, what are the various ports, some common ones; what “**Types of Protocols**” we have, namely **TCP & UDP**, difference between them; And last but not the least, “**Services Running**” at different ports with their respective protocols & uses.

## HTTP vs. HTTPS

### 1. HTTP vs HTTPS: What is the difference.

Ans: HTTPS is HTTP with encryption and verification. The only difference between the two protocols is that HTTPS uses TLS (SSL) to encrypt normal HTTP requests and responses, and to digitally sign those requests and responses. As a result, HTTPS is far more secure than HTTP. A website that uses HTTP has `http://` in its URL, while a website that uses HTTPS has `https`



### 2. What is HTTP?

Ans: HTTP stands for Hypertext Transfer Protocol, and it is a [protocol](#) — or a prescribed order and syntax for presenting information — used for transferring data over a network. Most information that is sent over the Internet, including website content and API calls, uses the HTTP protocol. There are two main kinds of HTTP messages: requests and responses.

In the OSI model (see [What is the OSI model?](#)), HTTP is a [layer 7](#) protocol.

### 3. What is HTTPS?

Ans: HTTPS is the **secure** version of HTTP, the protocol used for transferring data between your browser and a website.

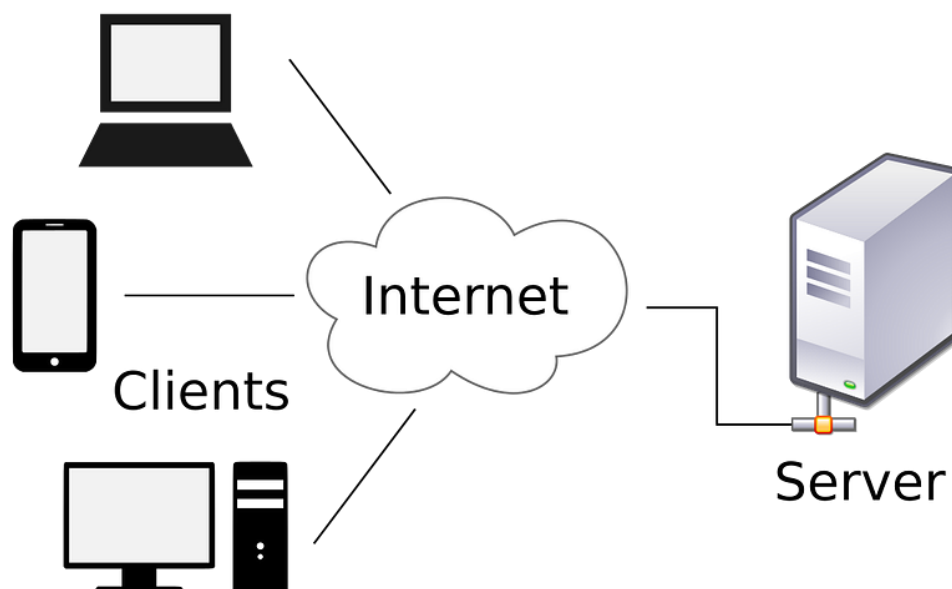
- a. It uses **SSL/TLS encryption** to secure the connection.
- b. Ensures **confidentiality, integrity, and authenticity** of the data.
- c. The URL starts with **https://** (instead of <http://>).

### Client Server Communication

What is Client-Server communication?

In client-server communication we have first of all two obvious partners: The **client** and the **server**.

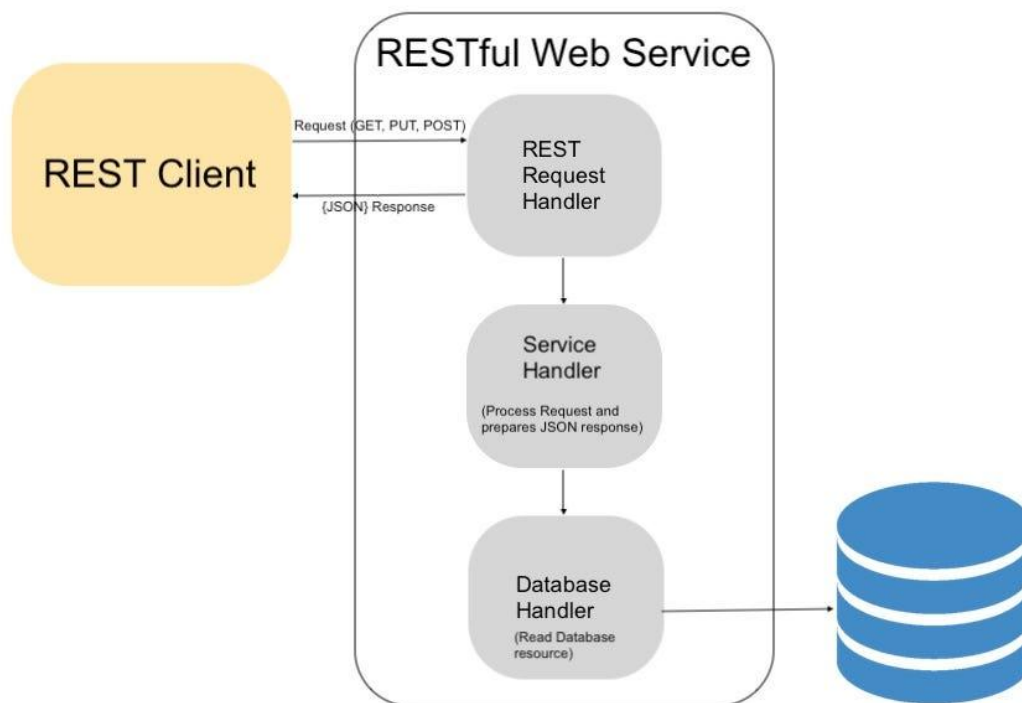
Press enter or click to view image in full size



To understand the communication between these two partners, we need to know some simple topics:

- **Requests:** Requests are sent from the client in order to ask the server for some data like files, or tell the server about things that happen, like that a user wants to login with his credentials
- **Response:** A response is sent from the server to the client and is the reaction of the server to a request of the client. This could for example be an authentication result.
- **Service:** A Service is a specific task that the server provides for the client to use, like downloading image

Press enter or click to view image in full size



When we need client-server communication?

Client-Server communication is always used, whenever someone requests something from the internet. Some examples would be applications like Facebook, Twitter, Instagram, Maps or even simple games like Temple Run and so on.

## REST

Let's first understand what is REST and RESTful Communication. Modern web services are often based on the REST architecture, which relies on the HTTP protocol and returns data in the JSON format.

What is RESTful Communication?

***RESTful** communication means implementation of a web service using **HTTP** and **REST** principles.*

### **REST Principles are:**

1. Application state and functionality are divided into resources and then the application would respond based on the resource that the operation is being performed
2. Resources are addressable using standard URIs that can be used as hypermedia links such as <http://example.com/resources/> and then for each resource the type of the resource is added in the URI such as <http://example.com/resources/resource>.
3. Resources provide information using MIME( Multipurpose Internet Mail Extensions)types supported by HTTP (e.g JSON, XML, Atom, etc.)
4. Use only standard HTTP methods, they are **GET**, **PUT**, **POST** and **DELETE**.
5. Protocol is stateless, cacheable and layered





**Stateless** which means that the server would not need to remember any kind of information regarding the state of the client whereas the client would need to include all the information in its request.

REST allows **caching** which means that the responses would need to be defined as cacheable which would eliminate some client-server interactions, further improving scalability and performance.

**Layered** meaning that the client does not know if the server that is responding is the actually the end server that is serving the resource which is a great principle to enable load balancing and provide shared caches.

**Note :** Most REST API implementations rely on agent driven content negotiations. **Agent driven content negotiation** rely on usage of HTTP request headers or resource URI patterns.

Content negotiation using HTTP headers

At server side, an incoming request may have an entity attached to it. To determine it's type, server uses the HTTP request header Content-Type. Some common examples of content types are "text/plain", "application/xml", "text/html", "application/json", "image/gif", and "image/jpeg"

## REST architecture

**REST is a client-server architecture.** The client and the server both have a different set of concerns. The server stores and/or manipulates information and makes it available to the user in an efficient manner. The client takes that information and displays it to the user and/or uses it to perform subsequent requests for information. This separation of concerns allows both the client and the server to evolve independently as it only requires that the interface stays the same.

**REST is stateless.** That means the communication between the client and the server always contains all the information needed to perform the request. There is no session state in the server, it is kept entirely on the client's side. If access to a resource requires authentication, then the client needs to authenticate itself with every request.

**REST is cacheable.** The client, the server and any intermediary components can all cache resources in order to improve performance.

**REST provides a uniform interface** between components. This simplifies the architecture, as all components follow the same rules to speak to one another. It also makes it easier to understand the interactions between the different components of the system.

**REST is a layered system.** Individual components cannot see beyond the immediate layer with which they are interacting. This means that a client connecting to an intermediate component, like a proxy, has no knowledge of what lies beyond. This allows components to be independent and thus easily replaceable or extendable.

## Resources and Resource identifiers

A resource is an abstract concept. In a REST system, any information that can be named may be a resource. This includes documents, images, a collection of resources and any other information. Any information that can be the target of an hypertext link can be a resource.

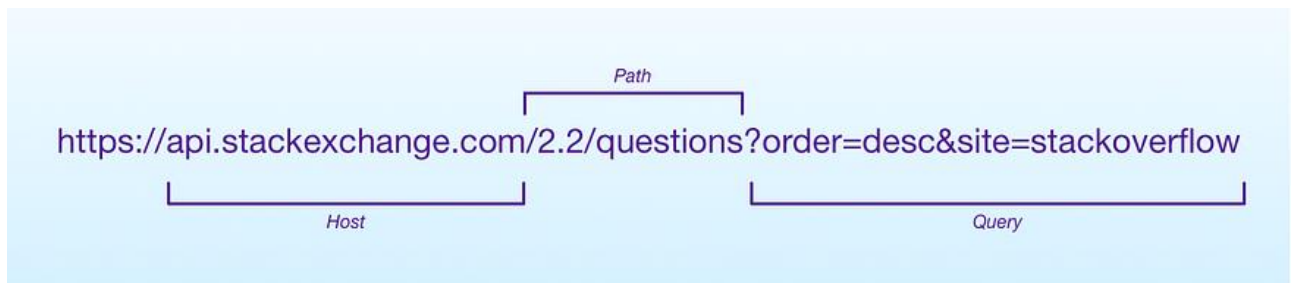
*A resource is a conceptual mapping to a set of entities.*

A resource is a conceptual mapping to a set of entities. The set of entities evolves over time; a resource doesn't. For example, a resource can map to “users who have logged in this past month” and another to “all users”. At some point in time they may map to the same set of entities, because all users logged in this past month. But they are still different resources. Similarly, if nobody logged in recently, then the first resource may map to the empty set. This resource exists regardless of the information it maps to.

***Resources are identified by uniform resource identifiers, also known as URIs.***

REST APIs use URLs and the HTTP protocol to identify resources and transfer data

Press enter or click to view image in full size



A URL has different parts, but in the context of REST APIs we are usually interested in just three:

- The host, which is usually a name (or sometimes the IP address) that identifies the other endpoint we are going to reach through the network
- A path, which identifies the resource we are looking for
- An optional query, to express extra parameters we might need for our request

These URLs, though, are just a part of what you need to understand to talk to a REST API.

REST is a type of architecture for web services. But as iOS developers, we don't care how the entire REST architecture works on the server side.

REST API calls using HTTP requests

REST works over the [Hypertext Transfer Protocol \(HTTP\)](#), which is a protocol originally created to allow the transmission of web pages over the internet.

In HTTP a session is usually a sequence of network requests and responses, although when making calls from iOS apps, we usually use a single session for every call we make.

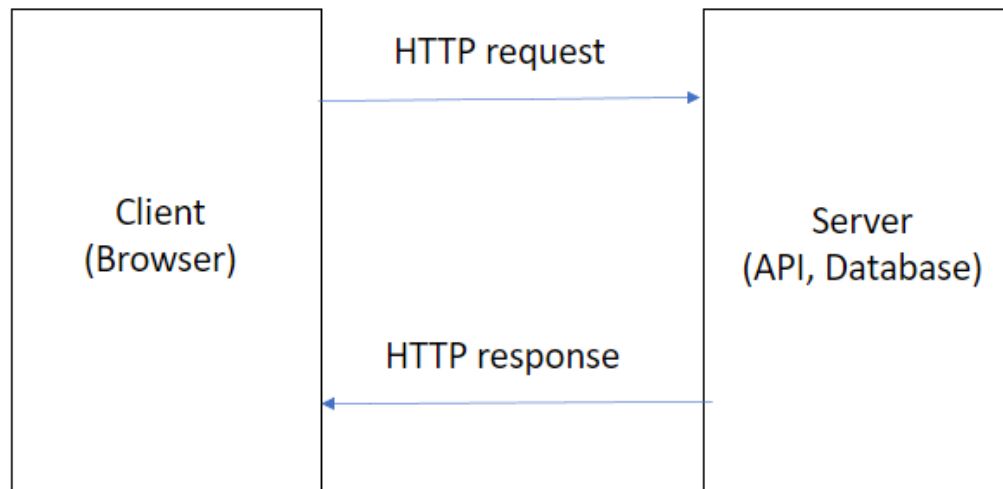
An HTTP request usually contains:

- a URL for the resource we want
- an HTTP method to state the action we want to perform
- optional parameters in the form of HTTP headers
- some optional data we want to send to the server

In a REST API call we use only a subset of the HTTP methods for the actions we need to perform:

- **GET**, to retrieve a resource
- **POST**, to create a resource
- **DELETE**, to delete a resource

**What is a HTTP Request/Response model?**



### 1. Client Sends a request:

- The client (usually a web browser or an application) initiates the communication by sending an HTTP request to a specific URL or endpoint on the server.
- The request includes the HTTP method (such as GET, POST, PUT, DELETE) that indicates the desired action to be performed on the server, along with additional headers and sometimes a message body containing data.

### 2. Server Parses the request:

- When the server receives a request from client, It needs to parse the request to understand the desired action needs to be performed. The request parsing step has many task to do (like extracting the request method, request Headers, determining start and end of a request, etc.)

### 3. Server processes the request:

- After parsing the request, Server knows the required action to perform, That action may performing various operations, such as retrieving data from a database, processing the data, or generating a response dynamically.

#### **4. Server sends the response:**

- After processing the request, the server formulates an appropriate response and sends it back to the client. The response includes the necessary information, such as status codes, headers, and a response body containing data or resources.

#### **5. Client parses the response and uses it:**

- Upon receiving the response, the client parses and processes it based on the provided status codes, headers, and message body. The client then utilizes the response to display results on a web page or perform further actions as needed.

#### **Where Request/Response Model is used ?**

The request-response model finds application in various network protocols and systems where clients and servers interact to exchange information. It is commonly used in APIs, network communications (e.g., SMTP), SSH, and other communication protocols.

#### **Anatomy Of Request**

Format of a HTTP Request

- # Request Line (GET, POST, ....)
- # Request Headers (User Agent, Host, etc.)
- # An empty line indicating the end of Header fields
- # Optional Message body

- **HTTP Methods:** GET, POST, PUT, DELETE, and more
- **Request Headers:** Metadata providing additional information about the request
- **Request Body:** Data sent by the client (**GET** HTTP verb doesn't have a body)

## Anatomy of Response

### Format of HTTP Response

- # Status Line
- # Response Headers (Additional info about response)
- # An empty line indicating the end of Header fields
- # Optional Message body

- **Status Codes:** Indicate the outcome of the request (e.g., 200 OK, 404 Not Found).
- **Response Headers:** Additional information that accompanies the response.
- **Response Body:** Data returned by the server (e.g., HTML, JSON).

## cookie headers

### What Are Cookie Headers?

In HTTP responses, cookies are set by servers using the **Set-Cookie** header, which instructs the browser to store the cookies for future requests. On subsequent requests, the browser sends the cookies back to the server using the **Cookie** header. These headers allow the server and browser to exchange data, such as user authentication or tracking information.

### 1. Set-Cookie Header

The `Set-Cookie` header is sent by the server to store cookies in the client's browser. It looks something like this:

```
Set-Cookie: sessionId=abc123; expires=Wed, 21 Oct 2025 07:28:00 GMT; path=/; secure;
HttpOnly
```

Each of these components plays a specific role:

- **sessionId=abc123**: The cookie's name (`sessionId`) and its value (`abc123`).
- **expires=Wed, 21 Oct 2025 07:28:00 GMT**: The expiration date for the cookie. If not set, the cookie will expire when the browser is closed (a session cookie).
- **path=**/: This determines the URL path for which the cookie is valid. It can be restricted to certain parts of the website or be global (i.e., /).
- **secure**: This flag means the cookie is only sent over HTTPS connections, ensuring it's encrypted.
- **HttpOnly**: Prevents the cookie from being accessed through JavaScript (protecting it from certain types of cross-site scripting attacks).

## 2. Cookie Header

Once the cookies are set, the browser will include them in the `Cookie` header in subsequent HTTP requests to the same domain:

```
Cookie: sessionId=abc123
```



The browser automatically attaches any relevant cookies to requests for the matching domain, which the server can then read to process the request appropriately. This is why cookies are vital for session management and user authentication.

## Key Attributes of Cookie Headers

There are several important attributes that influence the behavior of cookies. Here's a breakdown of the most common ones:

### Expires / Max-Age

- **expires:** Defines when a cookie will expire. After this time, the cookie will be deleted. If the `expires` attribute is not set, the cookie is a session cookie and will be deleted once the browser is closed.
- **max-age:** Similar to `expires`, but instead of a specific time, it defines the duration (in seconds) for which the cookie is valid after being set.

### Path

The `path` attribute specifies which URL paths the cookie should be sent to. For example, if `path=/admin`, the cookie will only be sent for requests made to URLs that start with `/admin`. By default, the path is set to the path of the page that created the cookie.

### Domain

This specifies which domain the cookie should be sent to. A cookie's domain can be more specific, like `www.example.com`, or broader, like `example.com`. If a cookie is set to a broader domain, it will be available to all subdomains within that domain.

### Secure

A cookie with the `secure` flag will only be sent over HTTPS connections. This ensures that the cookie's data is encrypted, preventing interception by malicious actors.

## HttpOnly

When this attribute is set, the cookie is inaccessible to JavaScript running on the page. This is particularly useful for protecting session cookies from cross-site scripting (XSS) attacks, as malicious scripts cannot access the cookie directly.

## SameSite

Introduced to enhance security, the `SameSite` attribute restricts how cookies are sent with cross-site requests. There are three possible values for `SameSite`:

- **Strict:** Cookies are only sent for same-site requests, preventing cross-site request forgery (CSRF) attacks.
- **Lax:** Cookies are sent for same-site requests and top-level navigations (e.g., clicking a link to the same site).
- **None:** Cookies are sent with all requests, but they must be set with the `Secure` flag.

## Why Are Cookie Headers Important?

Cookie headers are vital because they:

1. **Enable Session Management:** Cookies are often used to store session data for users once they log in. The `sessionId` in the cookie allows the server to recognize the user and provide them with personalized content.
2. **Improve User Experience:** Cookies can store user preferences, like language selection or items in a shopping cart, ensuring that when a user returns to the site, their preferences are remembered.
3. **Enhance Security:** Properly set cookies (with attributes like `Secure`, `HttpOnly`, and `SameSite`) can mitigate many common web vulnerabilities, such as XSS and CSRF attacks.

4. **Facilitate Tracking and Analytics:** Cookies are also used to track user interactions with websites. This data is valuable for web analytics and personalized marketing, though it should be used responsibly in compliance with data privacy laws like GDPR.

### Best Practices for Using Cookie Headers

While cookies are powerful tools, they need to be used correctly to ensure that they're secure, functional, and compliant with privacy regulations. Here are some best practices:

1. **Use HttpOnly and Secure for Sensitive Cookies:** Always set the HttpOnly and Secure flags on cookies that store sensitive information, such as session tokens. This minimizes the risk of XSS and man-in-the-middle attacks.
2. **Implement SameSite for CSRF Protection:** Set SameSite to Strict or Lax to prevent cookies from being sent in cross-site requests, adding an additional layer of security against CSRF attacks.
3. **Ensure Cookie Expiry is Set Correctly:** Always set the appropriate expiry or max-age for cookies. If the cookie is meant to persist beyond a single session, set an explicit expiry time. Session cookies should only live until the browser is closed.
4. **Comply with Privacy Regulations:** Ensure that cookie usage complies with privacy laws such as GDPR and CCPA. Users must be informed about the cookies used and consent must be obtained in certain cases.

### status codes:

HTTP status codes are three-digit numbers that indicate whether a request succeeded, failed, or requires additional steps. Understanding these codes is crucial for debugging issues, building reliable APIs, and creating better user experiences. There are 5 classes, including:

- 1xx informational responses
- 2xx success responses
- 3xx redirection responses

- 4xx client error responses
- 5xx server error responses

### **What are HTTP status codes?**

HTTP status codes are included in every HTTP response and communicate the outcome of a request. The first digit indicates the class of response, and the next two digits provide additional detail. In practice, they're a universal handshake between clients and servers: at a glance you know if the request worked, the client needs to fix something, or the server had a problem.

### **The five classes of HTTP status codes**

HTTP status codes are grouped into five classes, each beginning with a number that represents the type of response.

#### **1xx informational responses**

Informational status codes indicate that the server has received the request and is processing it. These codes are rarely seen in typical API interactions, but they play important roles in protocol-level communication.

- **100 Continue** signals that the initial part of the request has been received, and the client should proceed with sending the rest. This is useful when uploading large files, since it lets the server reject requests early if there's a problem with the headers.
- **101 Switching Protocols** tells the client that the server is changing the protocols as requested, such as upgrading from HTTP to WebSocket for real-time communication.
- **102 Processing** is an interim response that indicates the server is still working on the request but hasn't completed it yet. This prevents clients from timing out during long operations.

#### **2xx success responses**

Success codes confirm that the client's request was received, understood, and successfully processed. These are the responses you want to see when testing your API.

- **200 OK** indicates that the request was successful and the server returned the requested data.
- **201 Created** means the request succeeded and created a new resource. POST requests typically return this code along with a Location header pointing to the newly created resource.

- **202 Accepted** indicates the request has been accepted for processing but hasn't been completed yet. This status is common for asynchronous operations where immediate processing isn't possible.
- **204 No Content** confirms that the request was successful, but the server did not return any data. This is typical for successful DELETE operations or PUT requests that don't need to return data.

### **3xx redirection responses**

Redirection codes tell the client that additional action is needed to complete the request, usually involving navigation to a different URL.

- **301 Moved Permanently** indicates that the requested resource has permanently moved to a new URL. Clients should update their bookmarks and future requests to point to the new URL provided in the Location header.
- **302 Found** signals a temporary redirect. The resource is temporarily available at a different URL, but future requests should continue using the original location.
- **303 See Other** directs the client to retrieve the response using a GET request to the URL specified in the Location header. This is commonly used after POST requests to prevent duplicate submissions.
- **304 Not Modified** tells the client that the cached version is still valid. When a client sends a conditional GET request with headers like If-Modified-Since, this response indicates no download is necessary.
- **307 Temporary Redirect** is similar to 302, but guarantees the HTTP method won't change when following the redirect. If the original request was POST, the redirected request must also be POST.
- **308 Permanent Redirect** combines the permanence of 301 with the method preservation of 307, ensuring both the URL and HTTP method remain consistent across redirects.

### **4xx client error responses**

These codes indicate that there was an issue with the client's request, such as a mistyped URL or invalid credentials. These errors require changes from the client before the request can succeed.

- **400 Bad Request** means the server couldn't understand the request due to invalid syntax, malformed JSON, or missing required fields. This is a catch-all for requests that don't meet the API's requirements.
- **401 Unauthorized** signals that authentication is required and either wasn't provided or was invalid. Despite its name, this code specifically relates to authentication, not authorization.
- **403 Forbidden** indicates the client is authenticated but doesn't have permission to access the requested resource. This is the correct code for authorization failures.
- **404 Not Found** means the requested resource doesn't exist at the specified URL. This could be because the resource was deleted, never existed, or the URL is incorrect.
- **405 Method Not Allowed** tells the client that the HTTP method used isn't supported for this endpoint. For example, trying to POST to a read-only resource would trigger this error.
- **408 Request Timeout** indicates the server timed out waiting for the client to send the complete request. This can happen with slow network connections or when uploading large files.
- **409 Conflict** signals that the request conflicts with the current state of the server. This commonly occurs when trying to create a resource that already exists or when simultaneous updates cause conflicts.
- **410 Gone** means the resource previously existed but has been permanently removed and won't be available again. Unlike 404, this explicitly indicates the resource was intentionally deleted.
- **429 Too Many Requests** indicates the client has exceeded rate limits. APIs use this code to prevent abuse and manage server load, often including headers that specify when the client can retry.

### 5xx server error responses

Server error codes indicate problems on the server side. These errors mean the request itself was valid, but the server couldn't fulfill it due to an internal issue.

- **500 Internal Server Error** is a generic error indicating something went wrong on the server. This catch-all status code doesn't provide specifics but signals an unexpected condition that prevented the request from completing.
- **502 Bad Gateway** occurs when a server acting as a gateway or proxy receives an invalid response from an upstream server. This often happens in load-balanced environments or when using reverse proxies.

- **503 Service Unavailable** means the server is temporarily unable to handle requests, often due to maintenance or overload. The Retry-After header can indicate when the client should try again.
- **504 Gateway Timeout** indicates that a gateway or proxy server didn't receive a timely response from an upstream server. This is common in microservice architectures where services depend on each other.

